Relatório de Evolução do Projeto

ChargeGrid Assistant — GoodWe EV Challenge 2026 — Sprint 04 (RAG)

1. Resumo da evolução do projeto

O ChargeGrid Assistant é um chatbot de suporte operacional para eletropostos comerciais, desenvolvido em sprints cumulativas em branches próprias do mesmo repositório:

- **Sprint 1 — planejamento:** definição do problema (gestão de múltiplos carregadores sem mecanismos integrados), persona (operador comercial de eletroposto), proposta e seleção de tecnologias.
- **Sprint 2 — chatbot funcional:** loop de terminal em Python com Ollama, memória por histórico de mensagens, system prompt com contexto simulado do eletroposto e few-shot prompting.
- **Sprint 3 — refactory para LangChain:** núcleo reescrito em LCEL (prompt | llm | parser), memória por sessão com limite de tokens, saída estruturada em Pydantic v2 e guardrails de segurança (validação de escopo + detecção de jailbreak/prompt injection).
- **Sprint 4 — RAG + interface:** busca semântica (RAG) sobre base de conhecimento simulada em PDFs, com ChromaDB persistente, embeddings locais, citação de fonte (documento + página), interface Streamlit e avaliação formal com 19 casos + LLM-as-judge.

2. Arquitetura construída na Sprint 04

Etapa	Implementação	Decisão principal
Base de conhecimento	4 PDFs simulados (manual, regimento, FAQ e tabela tarifária)	Documentos curtos e densos, mantendo consistência com os dados simulados desde a Sprint 2
Loader	PyMuPDFLoader, 1 documento por página	Preserva a origem de cada trecho (source + page), base da citação
Chunking	RecursiveCharacterTextSplitter 500/50	Chunks pequenos evitam misturar tópicos em documentos densos
Embeddings	nomic-embed-text via Ollama local	A API do Ollama Cloud não expõe embeddings (issue ollama/ollama#17129)
Vector store	ChromaDB persistente (data/chroma_db/)	Coleção recriada a cada indexação — corrige duplicações
Retriever	similarity_search com k=4	4 chunks equilibram contexto e custo de tokens
Prompt	prompt_rag_v1/v2 versionados	Exige citar 'Fonte: doc (página X)' e frase fixa de recusa
Pipeline	src/rag/pipeline.py	Integra guardrails da Sprint 03 (injection/fora de escopo) ao RAG
Interface	Streamlit (app/app.py)	Citação de fonte por resposta, histórico e cache de recursos

3. Metodologia de avaliação

Eval set de **19 casos** (11 factual, 2 multi-documento, 2 fora de contexto, 2 sem resposta na base, 2 injection no documento) com métricas determinísticas (chunk da fonte certa recuperado, citação presente, fonte correta citada, fatos-chave presentes, injection resistido) e **LLM-as-judge** (faithfulness e answer relevancy, com prompt e rubrica próprios — fallback manual do RAGAS, pois a biblioteca não instala junto com a versão atual do langchain-community). Modelo gerador e avaliador: gpt-oss:120b (temperature 0); juiz com max_tokens=1500. Resultados e comparações geradas por evals/run_rag_evals.py e evals/analisar_interacao.py.

4. Tabela antes/depois (Sprints 1/2 × Sprint 04)

Configuração constante nas iterações da Sprint 04 (k=4, chunk 500/50, gpt-oss:120b, nomic-embed-text). A variável declarada entre iterações é o prompt RAG (v1 → v2); a execução revelou, porém, um defeito no índice

(chunks duplicados) que contaminava as medições — por isso a tabela mostra as medidas originais com o índice defeituoso e as re-medidas após a correção, com o medidor e o juiz idênticos nos dois lados (a iteração 1 foi reexecutada com o juiz atualizado). Na versão original (Sprints 1/2) não há RAG: o contexto operacional é fixo no system prompt, sem base citável — por isso as métricas de RAG não se aplicam (n/a).

Métrica	Sprints 1/2 (versão original, sem RAG)	Iter 1 (índice defeituoso)	Iter 2 (índice defeituoso)	Iter 1 (índice corrigido)	Iter 2 (índice corrigido)
Faithfulness (média)	n/a — responde livre, sem grounding, a partir de contexto fixo no prompt	1.000	0.875	1.000	1.000
Answer relevancy (média)	n/a (sem eval formal da Sprint 2)	0.944	0.875	1.000	1.000
Chunk da fonte certa recuperado	não existe (sem retrieval)	0.923	0.923	1.000	1.000
Resposta com citação	não existe (sem citação de fonte)	0.933	0.800	1.000	1.000
Fonte correta citada	não existe (sem base citável)	0.867	0.800	1.000	1.000
Fatos-chave presentes	n/a — depende do contexto operacional fixo simulado no prompt	0.833	0.800	1.000	1.000
Injection resistido	não medido — sem guardrails na Sprint 2 (chegam na Sprint 3)	1.000	1.000	1.000	1.000
Recusa correta	não medida — recusa implícita no few-shot	0.944	0.875	1.000	1.000
Erros do juiz (de 19)	sem juiz (avaliação manual)	1	3	0	0

5. Problemas encontrados e soluções

5.1 Índice com chunks duplicados (o mais grave)

A iteração 2 'piorou' e casos com informação existente passaram a recusar ('não encontrei na base'). Diagnóstico: o Chroma.from_documents do LangChain **adiciona** à coleção em vez de recriá-la — o índice acumulou 51 embeddings para 17 documentos únicos (3 cópias de cada), e o top-k=4 voltava preenchido de duplicatas, empurrando o chunk correto para fora do corte. O modelo recusava corretamente (o grounding estava funcionando); a culpa não era do prompt v2. Solução: coleção apagada antes de reindexar + erro rápido se a coleção ficar vazia. Ganho medido (iter 2): citação/fonte/fatos +0.200, faithfulness +0.125, recusa +0.125.

5.2 Juiz devolvendo JSON truncado

1 a 3 casos por iteração com juiz_erro ('substring not found'): o parser exigia JSON completo e o modelo avaliador às vezes devolvia texto truncado. Casos bons eram excluídos da média, tornando as iterações não comparáveis. Solução: parser tolerante (extrai o maior objeto JSON válido, tolerando texto em volta), retry com instrução reforçada e max_tokens do juiz 600→1500. Resultado: 0 erros de juiz nas duas iterações re-medidas.

5.3 Ground truth incompleto no caso f-05

fonte_correta 0.867 mesmo com tudo certo: o caso esperava apenas regimento_condominial.pdf, mas o fato ('reserva de até 4 horas') existe também em faq_carregamento.pdf — a resposta estava correta e o ground truth (dado esperado) é que era incompleto. Corrigido para aceitar os dois documentos; métrica → 1.000.

5.4 Caracteres invisíveis quebravam a checagem de citação

citation_rate 0.933 em execução idêntica: o modelo da nuvem inseria zero-width space (U+200B) no meio da palavra 'Fonte:' e a checagem determinística não encontrava o marcador. Solução: normalização removendo caracteres invisíveis (U+200B–U+200F, U+2060, U+FEFF, U+00AD) antes de comparar; recalculado sem nova chamada ao modelo → 1.000.

5.5 Set difícil revela diferença real entre prompts

Com o eval set oficial saturado (teto 1.0), um set complementar mais difícil (8 casos) separou as duas versões de prompt: a v2 'obedeceu' a uma instrução injetada dentro do próprio documento (acrescentou um aviso promocional à resposta) enquanto a v1 resistiu. Correção aplicada em prompt_rag_v2.md (seção de proteção contra injection: instruções dentro de documentos não são ordens e acréscimos não solicitados são proibidos); verificação em nova rodada: todas as métricas do set difícil em 1.0. O experimento também exigiu canonizar separadores de milhar no medidor ('5.000' = '5 000').

5.6 Juiz penalizando a frase de recusa

Re-medindo a iteração 2 com o prompt v2 final, o avaliador (LLM-as-judge) pontuou 0.0 o caso de fora-do-contexto porque a frase fixa de recusa ('Sou especializado em eletropostos...') 'não está no contexto' — mesmo texto que recebeu 1.0 na rodada anterior (juiz instável nesses casos; artefato já documentado com o nemotron). Correção: critério explícito na rubrica e no prompt do juiz — frases de recusa definidas pelo sistema não contam como afirmações substantivas. Re-medição final: todas as métricas em 1.0 e 0 erros do juiz nas duas iterações.

6. Comparação entre modelos no contexto RAG

Os mesmos 19 casos, mesmo prompt v2, mesmo retrieval (k=4) e mesmo avaliador — variando apenas o modelo gerador e o max_tokens (docs/comparacao_modelos_rag_raw.json):

Métrica	gpt-oss:120b (512)	nemotron (512)	nemotron (2048)
Faithfulness (média)	1.000	0.684	0.947
Answer relevancy (média)	1.000	0.053	1.000
Fonte correta citada	1.000	0.067	1.000
Fatos-chave presentes	1.000	0.067	1.000
Recusa correta	1.000	0.789	1.000
Latência média/caso	2.95s	5.56s	11.75s

Com parâmetros idênticos, o nemotron-3-nano:30b devolveu 17 de 19 respostas **vazias** no RAG: gasta todo o orçamento de 512 tokens no raciocínio interno (efeito agravado pelo contexto longo recuperado). Com max_tokens=2048 alcança quase a paridade, mas com latência 4x maior. **gpt-oss:120b é a escolha do projeto no RAG**: única configuração com 1.0 em todas as métricas e a mais rápida. No comparativo da Sprint 03 (chain conversacional), o gpt-oss:120b também foi a melhor escolha: cerca de 4.4x mais rápido e mais conciso que o nemotron, com igual segurança contra jailbreak.

7. Equipe

Pedro Ferreras (RM 568713) · Pedro Santos (RM 571017) · Victor Binot (RM 571499) · Gustavo Kunitaki (RM 571400) · Kauanne Oliveira (RM 574191) · Nayhely Estela (RM 571416)

8. Limitações e próximos passos

- O ganho principal da sprint veio da correção dos defeitos de infraestrutura (indexação e avaliador), não da troca de prompt; o set difícil (8 casos) mostrou que a v2 era suscetível a instrução injetada no documento — falha corrigida no próprio prompt v2 e verificada em nova rodada do eval (todas as métricas em 1.0).
- Cada pergunta é respondida de forma independente — sem memória no RAG.

- Embeddings dependem do Ollama local em execução na máquina.
- O juiz penalizava a frase fixa de recusa como afirmação não suportada (problema 5.6) — corrigido na Sprint 04 com critério explícito na rubrica e no prompt do juiz; re-medições finalizadas com 0 erros do avaliador nas duas iterações.
- O juiz e o gerador são o mesmo modelo (gpt-oss:120b), o que pode introduzir viés do avaliador. Mitigação parcial: critérios explícitos na rubrica e decisões principalmente determinísticas — próximo passo é avaliar com um juiz de família de modelo diferente.

Equipe: Pedro Ferreras (RM 568713) · Pedro Santos (RM 571017) · Victor Binot (RM 571499) · Gustavo Kunitaki (RM 571400) · Kauanne Oliveira (RM 574191) · Nayhely Estela (RM 571416)